RECALL

FIAP × TOTVS CHALLENGE · INTELIGÊNCIA CONVERSACIONAL

Recall
API RESTful

Documentação técnica — Sprint 4 · Domain Driven Design (Java). Plataforma que transforma reuniões de vendas e CS em inteligência acionável.

INTEGRANTES

Bernardo Moreira RM 564103
Pedro Batista RM 563220
Renan Jordão RM 560618
Larissa Shiba RM 560462

EQUIPE

Recall

SOLUÇÃO

Recall — Inteligência Conversacional

DISCIPLINA

Domain Driven Design — Java · 2026

Sumário

1 Objetivo e escopo do projeto

2 Principais funcionalidades

3 Arquitetura e padrões de projeto

4 Endpoints da API RESTful

4.1 Empresas · 4.2 Insights · 4.3 Arquivos · 4.4 Usuários · 4.5 Padrão de erro

5 Protótipo — telas implementadas

6 Modelo do banco de dados (DER e DDL)

7 Diagrama de classes

1 Objetivo e escopo do projeto

Todos os dias acontecem centenas de reuniões de vendas e Customer Success no ecossistema TOTVS. Dores do cliente, menções a concorrentes, sinais de cancelamento e oportunidades de expansão ficam apenas na memória de quem participou da chamada e se perdem quando ela termina.

O **Recall** resolve esse problema: recebe arquivos e transcrições de reuniões (PDF, CSV e XLSX), estrutura o contexto de cada empresa cliente e gera **insights categorizados** (risco de churn, upsell, ameaça de concorrente, feedback de produto) acompanhados de uma **task pronta para o Jira**, para que cada sinal vire uma ação.

Escopo desta entrega (Sprint 4)

Na Sprint 3 o domínio foi modelado em Java com persistência em memória e interface `JOptionPane`. Nesta sprint o mesmo domínio foi transformado em uma **API RESTful em Spring Boot** com persistência em **Oracle**, mantendo o polimorfismo das entidades.

- CRUD completo de **Empresas**, **Insights**, **Arquivos** e **Usuários** (GET, POST, PUT, DELETE).
- Camadas **Model** → **DAO** → **Service** → **Controller**, com DTOs de entrada e saída.
- Validações de negócio no Service e Bean Validation nos DTOs.
- Tratamento centralizado de erros com respostas JSON padronizadas (400, 404, 500).
- Documentação interativa via Swagger UI (`/swagger-ui.html`).

> **Fora do escopo:** autenticação/autorização por token e o pipeline de IA (LangGraph) que gera os insights automaticamente — ambos ficam na plataforma web do Recall. A API Java expõe o domínio para ser consumido por esse Front-End.

Stack técnica

Camada	Tecnologia
Linguagem / Build	Java 21 · Maven (wrapper)
Framework	Spring Boot 3.3 (Spring Web, Spring JDBC, Bean Validation)
Acesso a dados	`JdbcTemplate` com SQL explícito (sem JPA)
Banco de dados	Oracle Database (FIAP) · driver `ojdbc11`
Documentação	springdoc-openapi (Swagger UI)

2 Principais funcionalidades

Gestão de empresas (contexto)

Cadastro da empresa cliente com setor, resumo consolidado e inteligência em JSON. O detalhe da empresa agrega todos os seus arquivos e insights. Nome único e exclusão em cascata.

Insights de vendas e CS

Registro de insights por categoria (`CHURN_RISK` , `UPSELL` , `COMPETITOR_THREAT` , `OPERACIONAL` , `FEEDBACK_PRODUTO`) e criticidade (`BAIXA` , `MEDIA` , `ALTA`).

Geração de task Jira

Ao criar ou consultar um insight, a API devolve o campo `taskJira` gerado por `Insight.gerarTaskJira()` , pronto para ser enviado ao Jira/CRM.

Ingestão de arquivos

Cadastro de arquivos PDF, CSV e XLSX vinculados à empresa. Cada subclasse valida a própria extensão (`validarFormato()`) e implementa a conversão para texto (`converterParaTexto()`).

Usuários e controle de acesso

Usuários `ADMIN` , `MANAGER` e `VIEWER` . O polimorfismo de `podeCriarContexto()` define quem pode criar contexto (Viewer é somente leitura).

Validação e erros padronizados

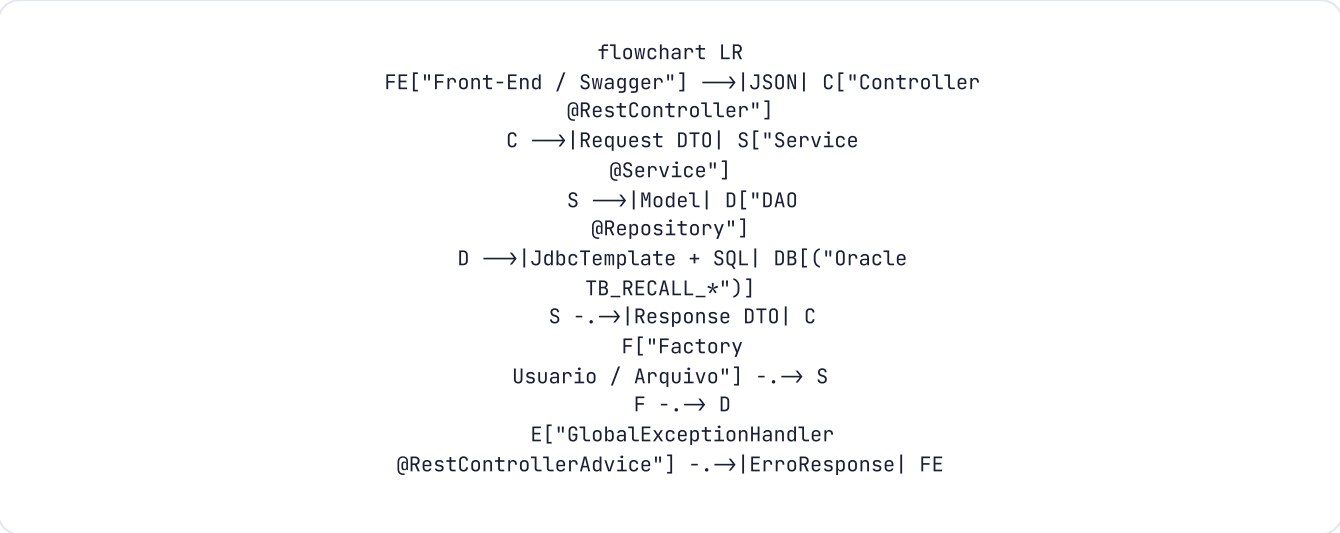
E-mail único e válido, empresa obrigatória para insights/arquivos, enums e extensões verificados. Todas as falhas retornam JSON com `status` , `error` , `message` e `path` .

Regras de negócio implementadas no Service

Entidade	Regra	Status
Empresa	`nome` e `industry` obrigatórios; não pode haver duas empresas com o mesmo nome.	`400`
Usuário	E-mail com formato válido e único; tipo deve ser ADMIN, MANAGER ou VIEWER.	`400`
Insight / Arquivo	A empresa informada na URI precisa existir.	`404`
Insight	Categoria e criticidade precisam ser valores válidos dos enums.	`400`
Arquivo	Tipo deve ser PDF, CSV ou XLSX e a extensão da URL deve ser compatível com o tipo.	`400`
Todos	Busca, atualização ou exclusão de ID inexistente.	`404`

3 Arquitetura e padrões de projeto

A API segue arquitetura em camadas. O Controller recebe e devolve apenas DTOs, o Service concentra validações e regras, e o DAO executa SQL no Oracle. O Controller nunca acessa o DAO e o DAO não conhece DTOs.



Padrões aplicados

Padrão	Onde e por quê
DAO	Interfaces em `dao/` e implementações em `dao/impl/` com `JdbcTemplate`. Isola o SQL do restante da aplicação.
DTO	`dto/request` (com `@NotBlank`, `@Email`, `@Size`) e `dto/response` (records). O model nunca é exposto diretamente.
Service	`service/` concentra as validações de negócio e a orquestração entre DAOs.
Factory	`UsuarioFactory` e `ArquivoFactory` instanciam a subclasse correta a partir do tipo — usadas no Service (criação) e no DAO (leitura do banco), preservando o polimorfismo.
Singleton	Beans Spring (`@Service`, `@Repository`, `@RestController`) têm escopo singleton por padrão.
Exception Handler	`GlobalExceptionHandler` traduz exceções de domínio em respostas HTTP padronizadas.
Injeção de dependência	Todas as dependências são recebidas pelo construtor.

Estrutura de pacotes

```
br.com.fiap.recall
├── config/       WebConfig (CORS + OpenAPI)
├── controller/   EmpresaController, InsightController, ArquivoController, UsuarioController
├── service/      EmpresaService, InsightService, ArquivoService, UsuarioService
├── dao/          EmpresaDao, InsightDao, ArquivoDao, UsuarioDao
│   └── impl/     *DaoImpl (JdbcTemplate)
├── dto/request/  EmpresaRequest, InsightRequest, ArquivoRequest, UsuarioRequest
├── dto/response/ EmpresaResponse, InsightResponse, ArquivoResponse, UsuarioResponse
├── factory/      UsuarioFactory, ArquivoFactory
├── model/        Empresa, Insight, Usuario(Admin/Manager/Viewer), Arquivo(PDF/CSV/XLSX), Categoria,
Criticidade
└── exception/    EntidadeNaoEncontradaException, RegraNegocioException, ErroResponse,
GlobalExceptionHandler
```

4 Endpoints da API RESTful

Base URL local: `http://localhost:8080`. Todos os endpoints consomem e produzem `application/json`. Os códigos de sucesso estão destacados em verde; qualquer endpoint pode retornar `500` em caso de falha inesperada.

Verbo	URI	Descrição	Status
4.1 Empresas			
GET	`/api/empresas`	Lista todas as empresas	200 · 500
GET	`/api/empresas/{id}`	Detalhe com arquivos e insights	200 · 404 · 500
POST	`/api/empresas`	Cadastra empresa	201 · 400 · 500
PUT	`/api/empresas/{id}`	Atualiza empresa	200 · 400 · 404 · 500
DELETE	`/api/empresas/{id}`	Exclui empresa (cascata em arquivos e insights)	204 · 404 · 500
4.2 Insights			
GET	`/api/empresas/{empresaId}/insights`	Lista insights da empresa	200 · 404 · 500
GET	`/api/insights/{id}`	Busca insight por ID	200 · 404 · 500
POST	`/api/empresas/{empresaId}/insights`	Cria insight e devolve a task Jira	201 · 400 · 404 · 500
PUT	`/api/insights/{id}`	Atualiza dados / criticidade	200 · 400 · 404 · 500
DELETE	`/api/insights/{id}`	Remove insight	204 · 404 · 500
4.3 Arquivos			
GET	`/api/empresas/{empresaId}/arquivos`	Lista arquivos da empresa	200 · 404 · 500
GET	`/api/arquivos/{id}`	Busca arquivo por ID	200 · 404 · 500
POST	`/api/empresas/{empresaId}/arquivos`	Cadastra arquivo e converte para texto	201 · 400 · 404 · 500
PUT	`/api/arquivos/{id}`	Atualiza URL / tipo	200 · 400 · 404 · 500
DELETE	`/api/arquivos/{id}`	Remove arquivo	204 · 404 · 500
4.4 Usuários			
GET	`/api/usuarios`	Lista usuários	200 · 500
GET	`/api/usuarios/{id}`	Busca usuário por ID	200 · 404 · 500

Verbo	URI	Descrição	Status
POST	`/api/usuarios`	Cadastra ADMIN, MANAGER ou VIEWER	201 · 400 · 500
PUT	`/api/usuarios/{id}`	Atualiza usuário	200 · 400 · 404 · 500
DELETE	`/api/usuarios/{id}`	Remove usuário	204 · 404 · 500

Significado dos códigos de status

Código	Quando ocorre
200 OK	Consulta ou atualização realizada com sucesso.
201 Created	Recurso criado; o corpo traz o recurso com o ID gerado (UUID).
204 No Content	Exclusão realizada; sem corpo de resposta.
400 Bad Request	Campo obrigatório ausente, formato inválido ou regra de negócio violada.
404 Not Found	ID do recurso (ou da empresa na URI) não existe.
500 Internal Server Error	Falha inesperada; a mensagem é genérica e não expõe stack trace.

4.5 Exemplos de requisição e resposta

```
// POST /api/empresas/{empresaId}/insights
{
  "categoria": "CHURN_RISK",
  "criticidade": "ALTA",
  "titulo": "Insatisfação com tempo de resposta
do suporte",
  "insight": "3 chamados críticos sem SLA
cumprido.",
  "acaoSugerida": "Agendar reunião com CS."
}

// 201 Created
{
  "id": "7c1e…",
  "categoria": "CHURN_RISK",
  "criticidade": "ALTA",
  "taskJira": "[JIRA-ALTA] [CHURN_RISK]
Insatisfação
    com tempo de resposta do suporte | Ação:
Agendar
    reunião com CS.",
  "empresaId": "e4a2…",
  "dataCriacao": "2026-10-20T16:43:36"
}
```

```
// GET /api/empresas/id-inexistente → 404
{
  "timestamp": "2026-10-20T16:43:38",
  "status": 404,
  "error": "Not Found",
  "message": "Empresa com id 'id-inexistente'
    não encontrada.",
  "path": "/api/empresas/id-inexistente"
}

// POST /api/usuarios (e-mail duplicado) → 400
{
  "status": 400,
  "error": "Bad Request",
  "message": "Já existe um usuário cadastrado com
o e-mail 'bernardo@recall.app'.",
  "path": "/api/usuarios"
}
```

5 Protótipo — telas implementadas

As telas abaixo foram capturadas da aplicação em execução. A plataforma web do Recall (Next.js, publicada na Vercel) é o Front-End da solução; a API Java é exercitada pela interface Swagger UI gerada automaticamente pelo springdoc-openapi, com dados reais persistidos no Oracle.

5.1 Plataforma web Recall

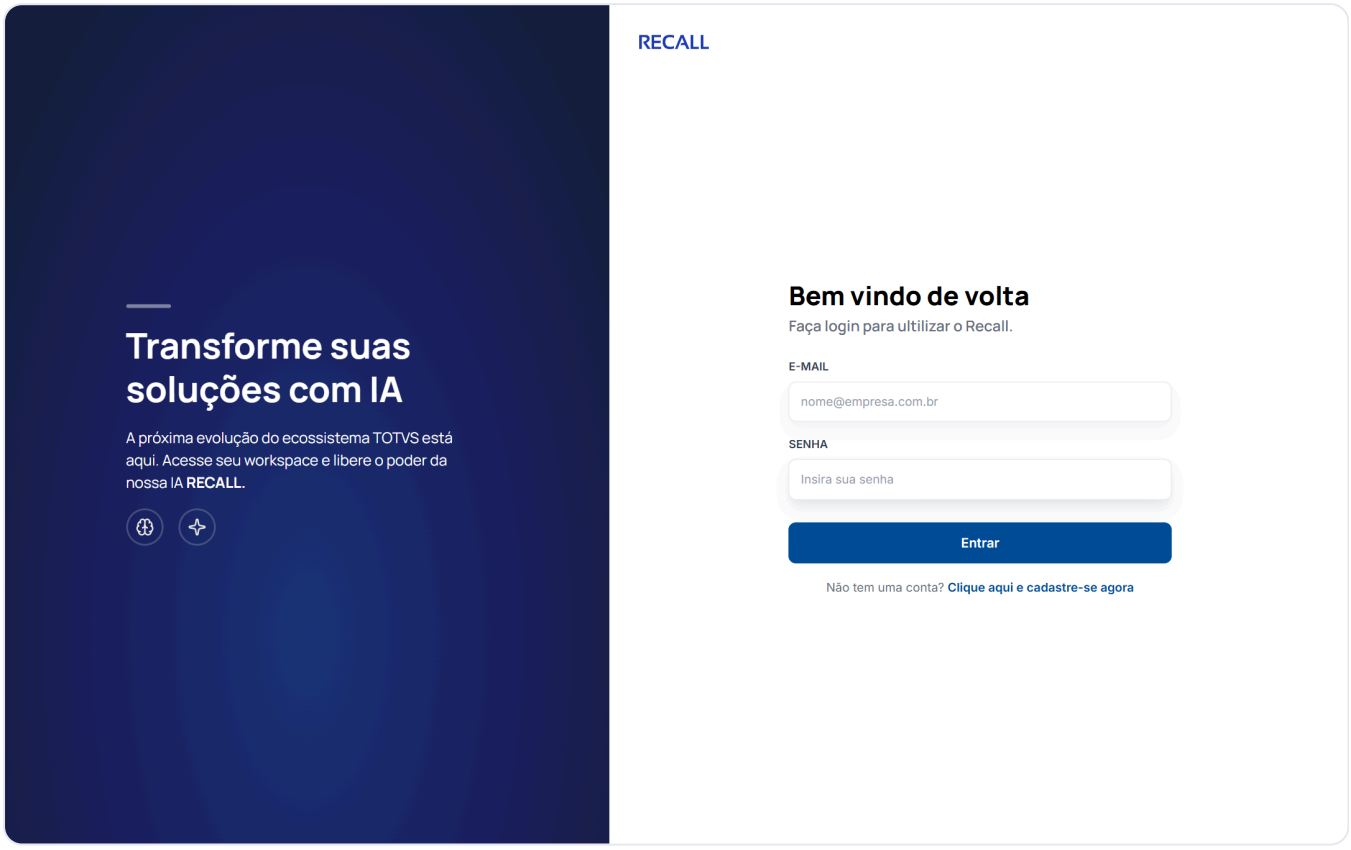


Figura 1 — Tela de login da plataforma Recall (recall-ten-delta.vercel.app)

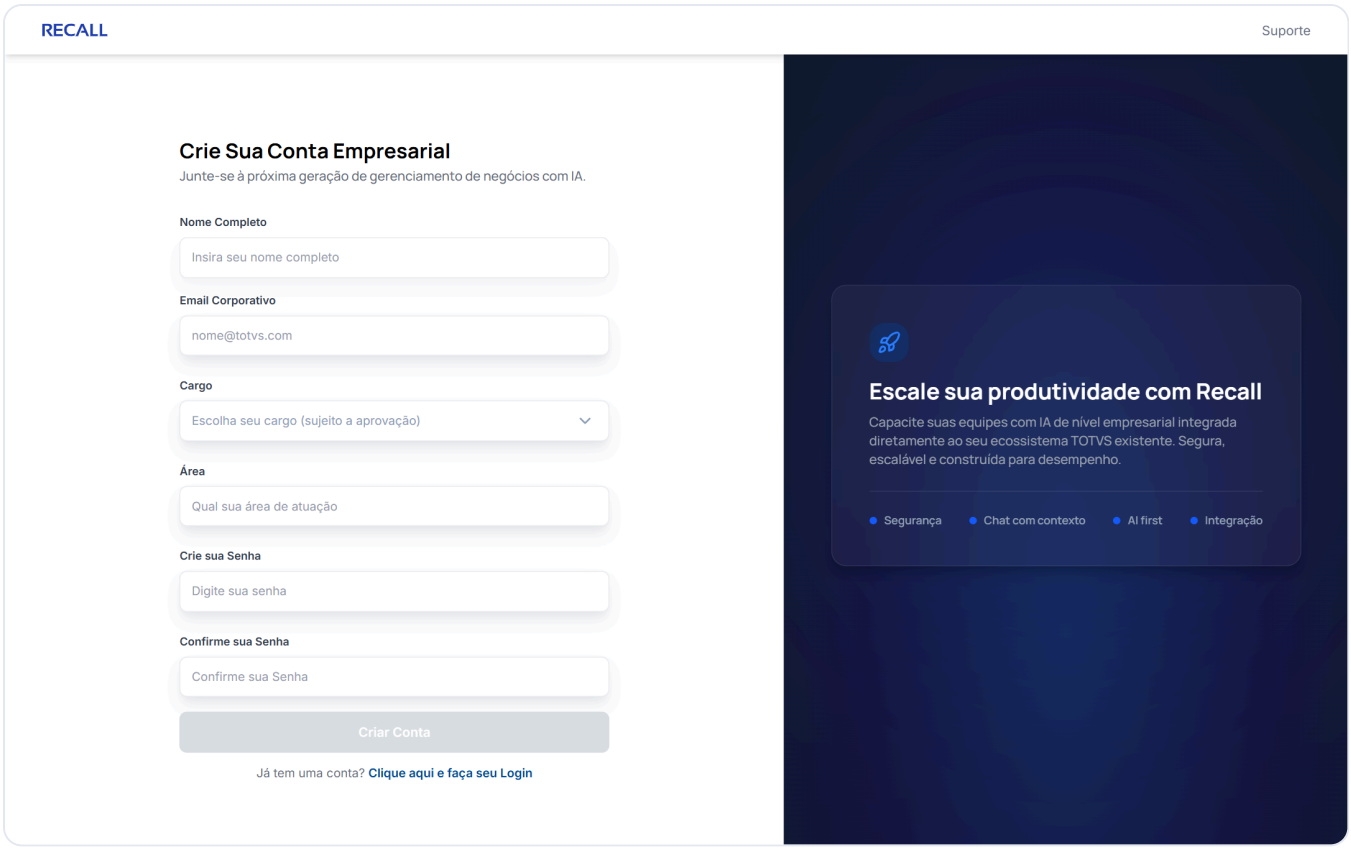


Figura 2 — Tela de cadastro de usuário

5.2 API Recall — Swagger UI

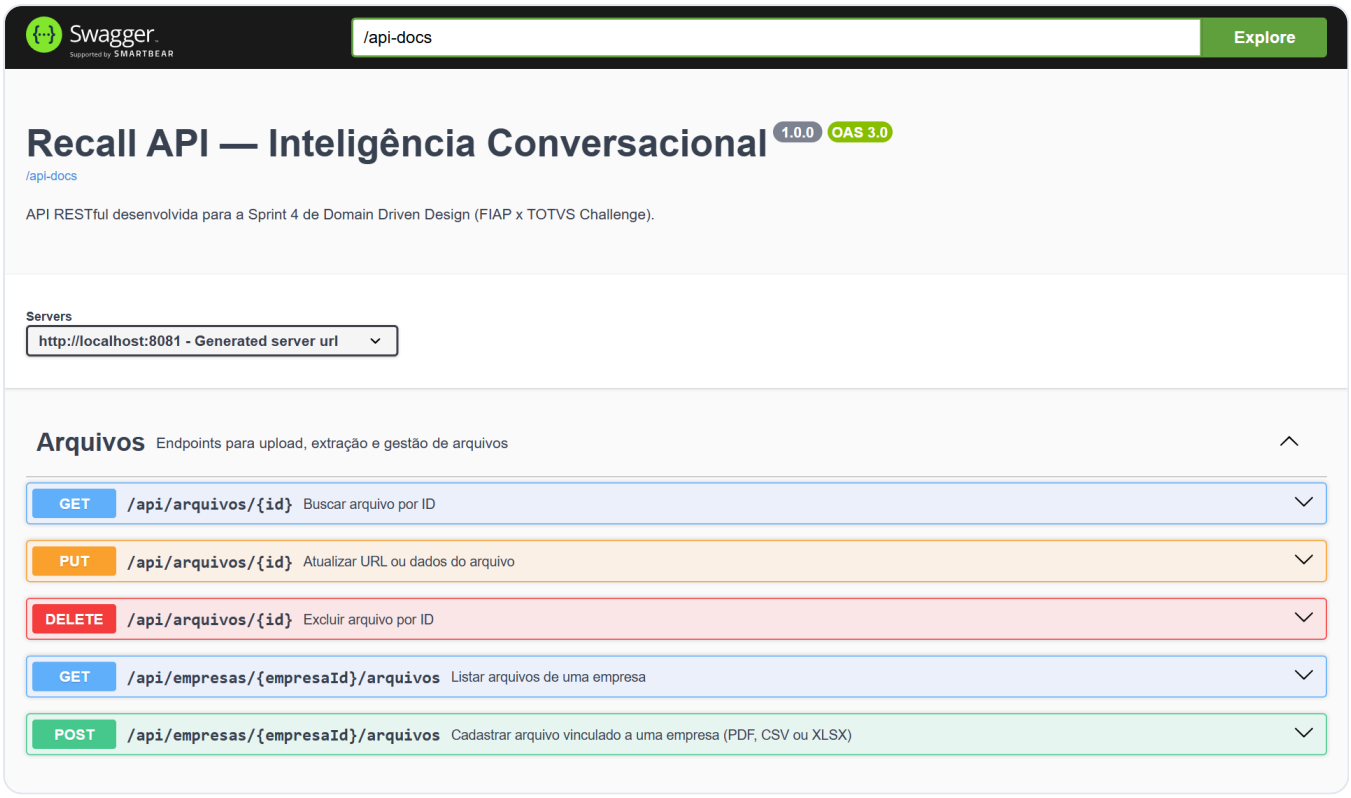


Figura 3 — Swagger UI com os grupos de endpoints da API (Empresas, Insights, Arquivos, Usuários)

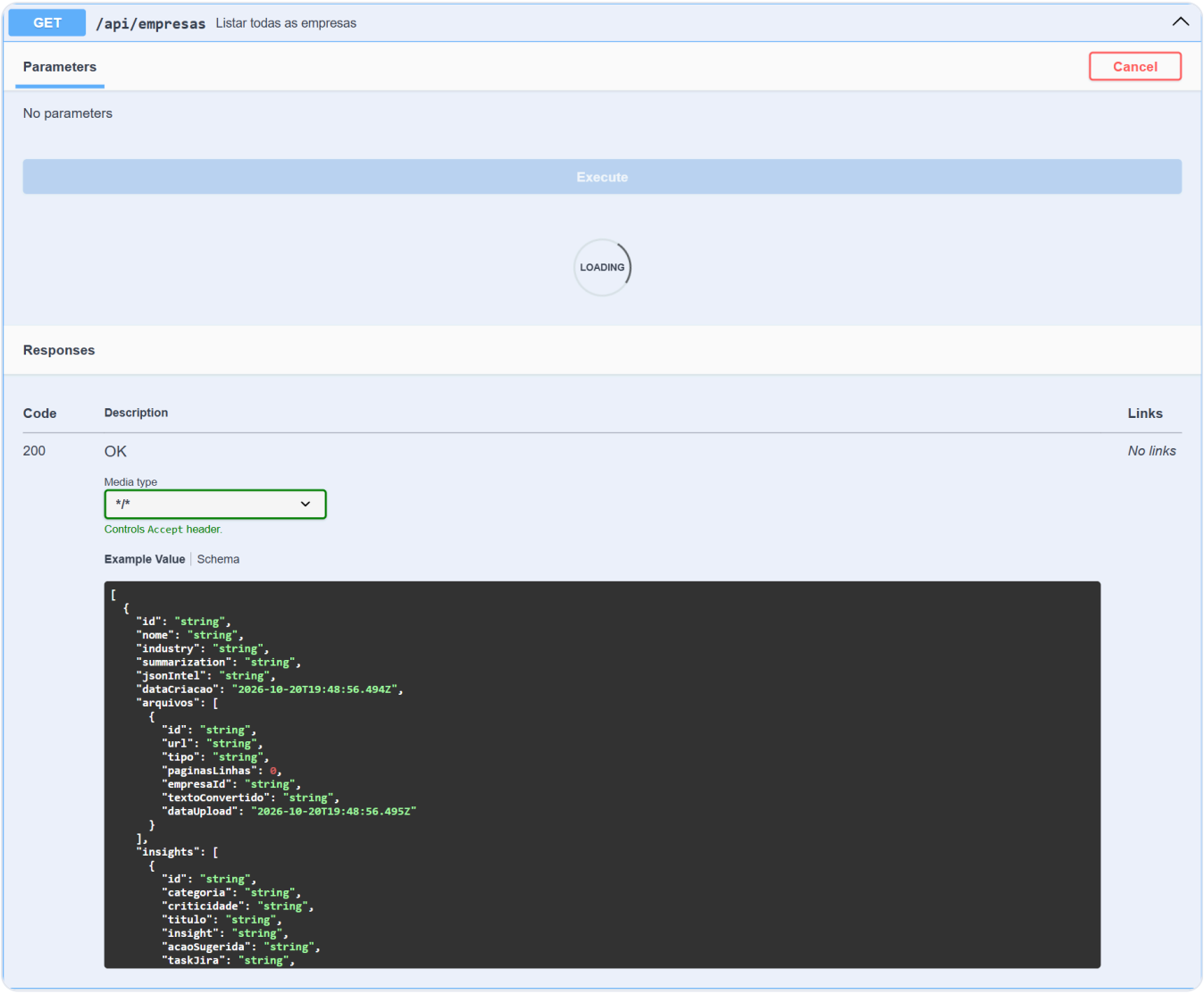


Figura 4 — `GET /api/empresas` retornando 200 com empresas persistidas no Oracle

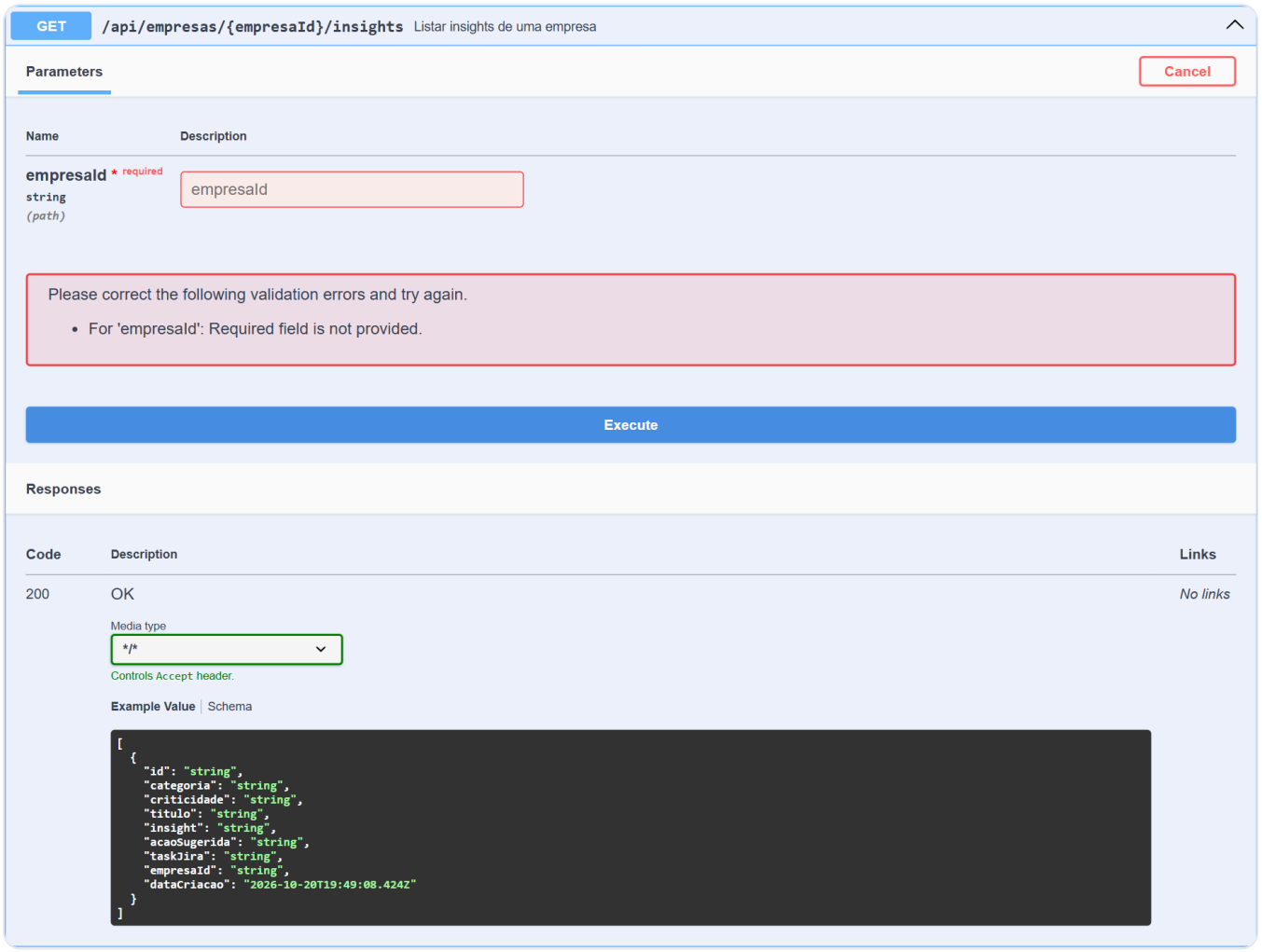


Figura 5 — `POST /api/empresas/{empresaId}/insights` retornando 201 com a task Jira gerada

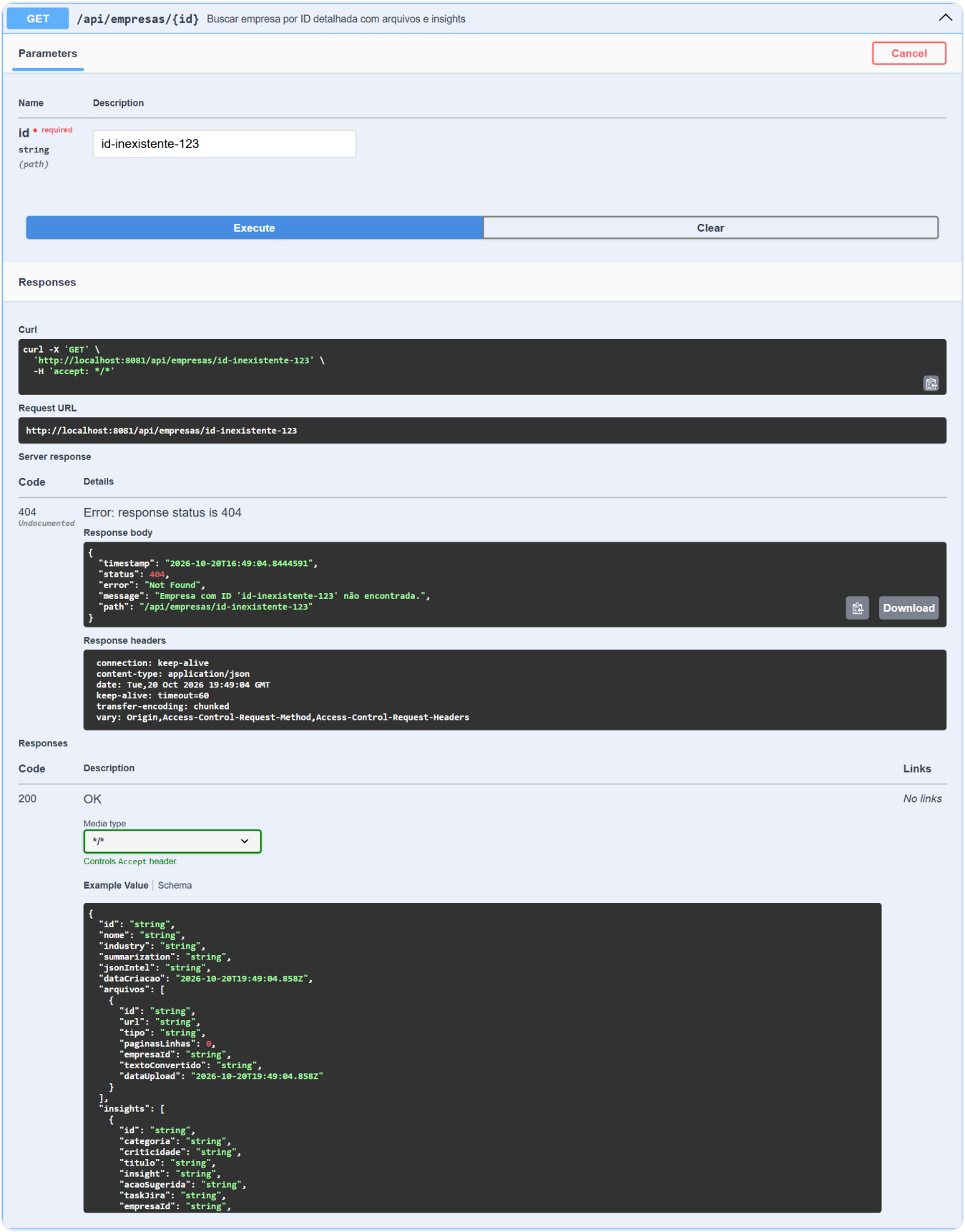


Figura 6 — Tratamento de erro: `GET /api/empresas/{id}` com ID inexistente retorna 404 no padrão `ErroResponse`

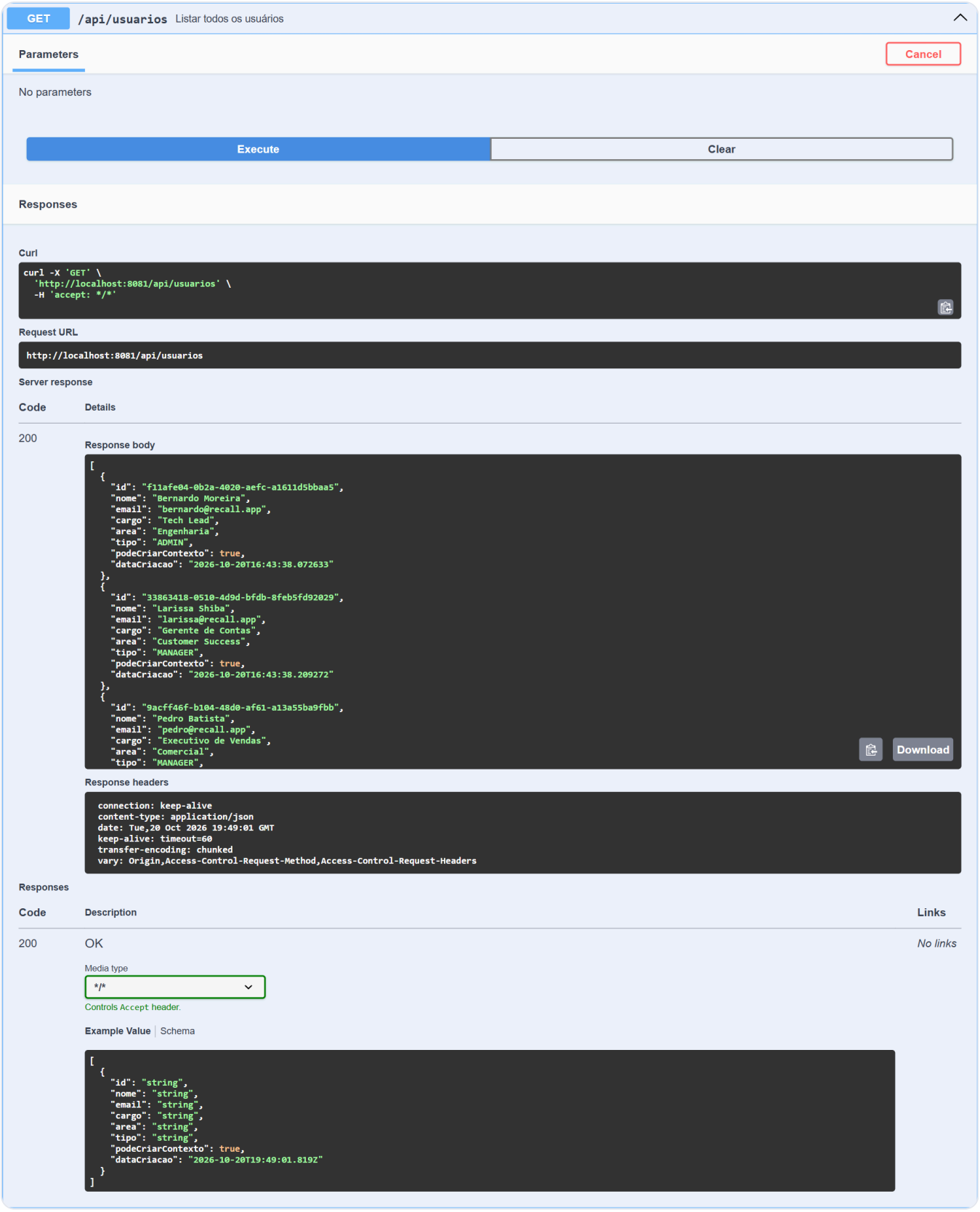


Figura 7 — `GET /api/usuarios`: polimorfismo de `podeCriarContexto()` por tipo de usuário

6 Modelo do banco de dados

O banco Oracle possui quatro tabelas com prefixo `TB_RECALL_`. `TB_RECALL_EMPRESA` é o agregador central: arquivos e insights pertencem a uma empresa e são removidos em cascata (`ON DELETE CASCADE`). O tipo das subclasses polimórficas é persistido em `TP_USUARIO` e `TP_ARQUIVO` (estratégia *single table*), e as Factories reconstroem a subclasse correta na leitura.

6.1 Diagrama Entidade-Relacionamento

```
erDiagram
TB_RECALL_EMPRESA |├-o{ TB_RECALL_ARQUIVO : "possui"
TB_RECALL_EMPRESA |├-o{ TB_RECALL_INSIGHT : "gera"
TB_RECALL_EMPRESA {
VARCHAR2_36 ID_EMPRESA PK
VARCHAR2_120 NM_EMPRESA "NOT NULL"
VARCHAR2_80 DS_INDUSTRY "NOT NULL"
CLOB DS_SUMMARIZATION
CLOB DS_JSON_INTEL
TIMESTAMP DT_CRIACAO "NOT NULL"
}
TB_RECALL_ARQUIVO {
VARCHAR2_36 ID_ARQUIVO PK
VARCHAR2_500 DS_URL "NOT NULL"
VARCHAR2_20 TP_ARQUIVO "PDF, CSV, XLSX"
NUMBER NR_PAGINAS_LINHAS
VARCHAR2_36 ID_EMPRESA FK
TIMESTAMP DT_UPLOAD "NOT NULL"
}
TB_RECALL_INSIGHT {
VARCHAR2_36 ID_INSIGHT PK
VARCHAR2_40 DS_CATEGORIA "NOT NULL"
VARCHAR2_20 DS_CRITICIDADE "NOT NULL"
VARCHAR2_150 NM_TITULO "NOT NULL"
VARCHAR2_1000 DS_INSIGHT
VARCHAR2_500 DS_ACAO_SUGERIDA
VARCHAR2_36 ID_EMPRESA FK
TIMESTAMP DT_CRIACAO "NOT NULL"
}
TB_RECALL_USUARIO {
VARCHAR2_36 ID_USUARIO PK
VARCHAR2_100 NM_USUARIO "NOT NULL"
VARCHAR2_120 DS_EMAIL "UNIQUE"
VARCHAR2_50 DS_CARGO "NOT NULL"
VARCHAR2_50 DS_AREA "NOT NULL"
VARCHAR2_20 TP_USUARIO "ADMIN, MANAGER, VIEWER"
TIMESTAMP DT_CRIACAO "NOT NULL"
}
```

6.2 Script DDL (`src/main/resources/schema.sql`)

```
CREATE TABLE TB_RECALL_USUARIO (
    ID_USUARIO   VARCHAR2(36) PRIMARY KEY,
    NM_USUARIO   VARCHAR2(100) NOT NULL,
    DS_EMAIL     VARCHAR2(120) NOT NULL UNIQUE,
    DS_CARGO     VARCHAR2(50) NOT NULL,
    DS_AREA      VARCHAR2(50) NOT NULL,
    TP_USUARIO   VARCHAR2(20) NOT NULL,
    DT_CRIACAO   TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL
);

CREATE TABLE TB_RECALL_EMPRESA (
    ID_EMPRESA        VARCHAR2(36) PRIMARY KEY,
    NM_EMPRESA        VARCHAR2(120) NOT NULL,
    DS_INDUSTRY       VARCHAR2(80) NOT NULL,
    DS_SUMMARIZATION  CLOB,
    DS_JSON_INTEL     CLOB,
    DT_CRIACAO        TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL
);

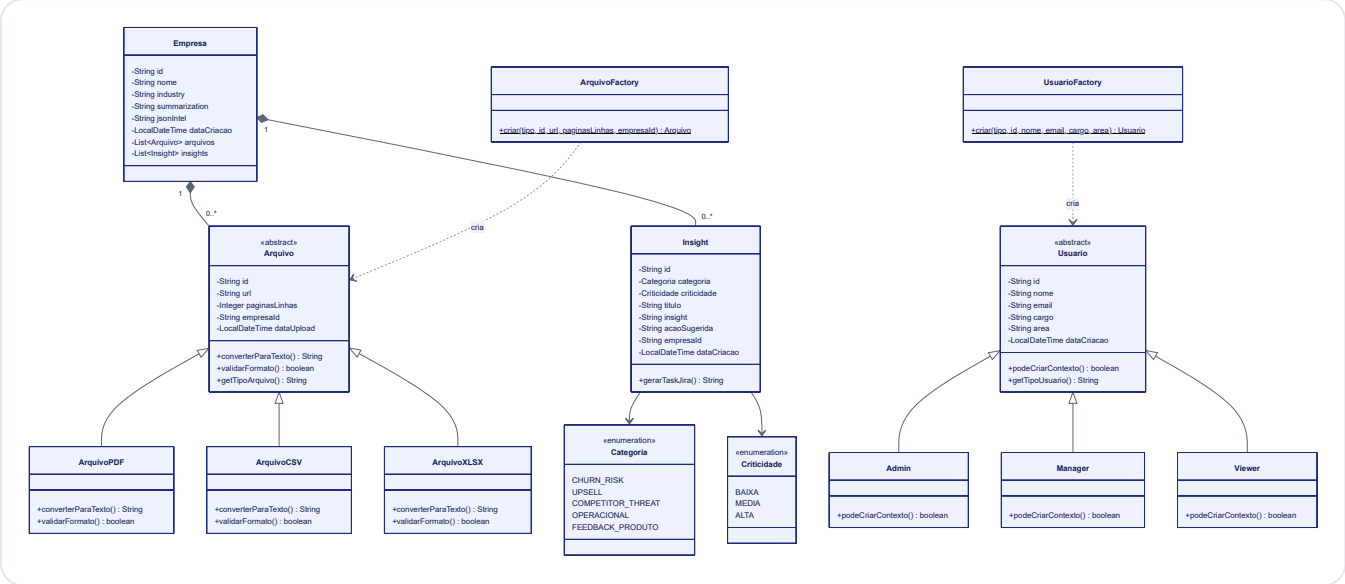
CREATE TABLE TB_RECALL_ARQUIVO (
    ID_ARQUIVO         VARCHAR2(36) PRIMARY KEY,
    DS_URL             VARCHAR2(500) NOT NULL,
    TP_ARQUIVO         VARCHAR2(20) NOT NULL,
    NR_PAGINAS_LINHAS  NUMBER,
    ID_EMPRESA         VARCHAR2(36) NOT NULL,
    DT_UPLOAD          TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
    CONSTRAINT FK_ARQUIVO_EMPRESA FOREIGN KEY (ID_EMPRESA)
        REFERENCES TB_RECALL_EMPRESA(ID_EMPRESA) ON DELETE CASCADE
);

CREATE TABLE TB_RECALL_INSIGHT (
    ID_INSIGHT        VARCHAR2(36) PRIMARY KEY,
    DS_CATEGORIA      VARCHAR2(40) NOT NULL,
    DS_CRITICIDADE    VARCHAR2(20) NOT NULL,
    NM_TITULO         VARCHAR2(150) NOT NULL,
    DS_INSIGHT        VARCHAR2(1000),
    DS_ACAO_SUGERIDA  VARCHAR2(500),
    ID_EMPRESA        VARCHAR2(36) NOT NULL,
    DT_CRIACAO        TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
    CONSTRAINT FK_INSIGHT_EMPRESA FOREIGN KEY (ID_EMPRESA)
        REFERENCES TB_RECALL_EMPRESA(ID_EMPRESA) ON DELETE CASCADE
);
```

7 Diagrama de classes

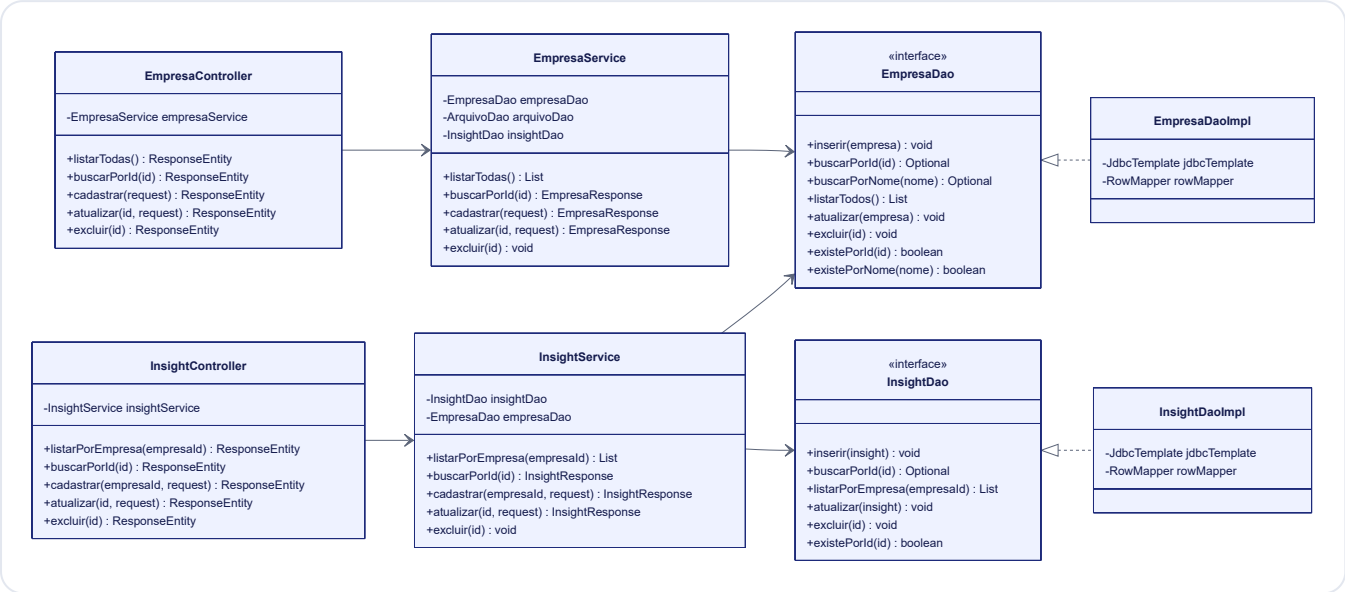
Diagrama atualizado em relação à Sprint 3: inclui os enums, as Factories e a remoção dos atributos sem correspondência no banco (`nivelAcesso`, `equipe`, `empresasVisiveis`). O polimorfismo permanece no comportamento: `podeCriarContexto()` em `Usuario` e `converterParaTexto()` / `validarFormato()` em `Arquivo`. Getters e setters foram omitidos.

7.1 Camada de domínio (model + factory)



7.2 Camadas de aplicação (Controller → Service → DAO)

Cada entidade possui o mesmo conjunto de classes; abaixo o fluxo completo de **Empresa** e **Insight**. `ArquivoController/Service/Dao` e `UsuarioController/Service/Dao` seguem a mesma estrutura e assinaturas equivalentes.



7.3 DTOs e tratamento de exceções

